



**Indian Institute of Information Technology,
Design and Manufacturing, Kancheepuram**

Traffic Signal Optimization using Reinforcement Learning

Team Members

Rohit Kumar (CS23B2053)
Gyan Chandra (CS23I1053)
Sarvan Kumar (ME23B1065)

Course Instructor

Dr. Rahul Raman

May 9, 2026

Contents

1	Introduction	2
2	Environment Design	2
2.1	Simulation Platform	2
2.2	Intersection Layout	2
2.3	Controller–Simulator Interface	3
3	Mathematical MDP Formulation	3
3.1	State Space	4
3.2	Action Space	4
3.3	Reward Function	4
3.4	Discount Factor and Episode Length	4
4	Reward Function Optimisation (Ablation Study)	5
4.1	Ablation Scenarios and Results	5
4.2	Analysis and Chosen Configuration	5
5	Control Strategies	5
5.1	Fixed-Time Controller (Baseline)	5
5.2	Q-Learning	6
5.3	Double Q-Learning	7
5.4	Deep Q-Network (DQN)	8
5.5	Advantage Actor-Critic (A2C)	9
5.6	Proximal Policy Optimization (PPO)	10
5.7	Trust Region Policy Optimization (TRPO)	11
6	Experimental Results	13
6.1	Training Convergence	13
6.2	Final Performance Summary	13
6.3	Improvement Over Fixed-Time Baseline	14
6.4	Metric Distribution Across Episodes	15
6.5	Discussion of Results	15
7	Conclusion	16
7.1	Summary of Findings	16
7.2	Limitations and Future Work	16
7.3	Closing Remarks	17

1 Introduction

Traffic congestion at urban intersections is one of the most persistent and costly challenges in modern city management. Poorly optimised signal timings cause cascading bottlenecks that increase fuel consumption, vehicular emissions, and driver frustration. Traditional **fixed-time controllers** employ static, pre-programmed phase sequences that cannot respond to real-time fluctuations in traffic demand — a critical limitation in Indian urban environments where traffic volumes are highly irregular.

Reinforcement Learning (RL) offers a principled, data-driven alternative. An RL agent learns an optimal signal-control policy through continuous trial-and-error interaction with a simulated traffic environment. By observing the traffic state (queue lengths, waiting times, active phase) and receiving scalar reward feedback, the agent autonomously learns when to hold, skip, or transition phases.

This project provides a comprehensive comparative evaluation of seven control strategies:

- **Fixed-Time Controller** — deterministic baseline.
- **Q-Learning** and **Double Q-Learning** — tabular value-based methods.
- **Deep Q-Network (DQN)** — neural function approximation.
- **A2C, PPO, and TRPO** — policy-gradient methods.

Before large-scale 300-episode training, a structured **Reward Function Ablation Study** was conducted over 50 episodes to mathematically optimise the environment feedback signal. The goal is to identify the most effective strategy for minimising congestion and maximising vehicle throughput at a realistic Indian-style four-way intersection simulated in SUMO.

2 Environment Design

2.1 Simulation Platform

The simulation is built on **SUMO (Simulation of Urban MObility)**, an open-source, microscopic traffic simulator capable of modelling individual vehicle dynamics, lane-change behaviour, car-following models, and signal-phase interactions at fine temporal resolution (1-second steps). The RL controller communicates with SUMO via the **TraCI (Traffic Control Interface)** API, enabling real-time reading of detector data and writing of signal commands at each decision step.

2.2 Intersection Layout

The modelled environment is a **four-way urban intersection** configured for Indian Left-Hand Drive (LHD) traffic rules with **two lanes per arm** (lane 0: straight + right-turn; lane 1: protected left-turn). Key design choices include:

- Realistic vehicle arrival rates sampled from Indian peak-hour distributions.
- Mixed traffic composition including cars, two-wheelers, and auto-rickshaws, defined via a `vtypes.add.xml` vehicle-type file.
- A **five-phase signal plan**: four active green phases (NS-straight, NS-left, EW-straight, EW-left) plus one implicit all-red clearance.
- A mandatory **4-second yellow** transition injected before every phase change to prevent vehicle collisions in the simulation.
- A **minimum green duration of 10 seconds** enforced to prevent unrealistic flickering between phases.

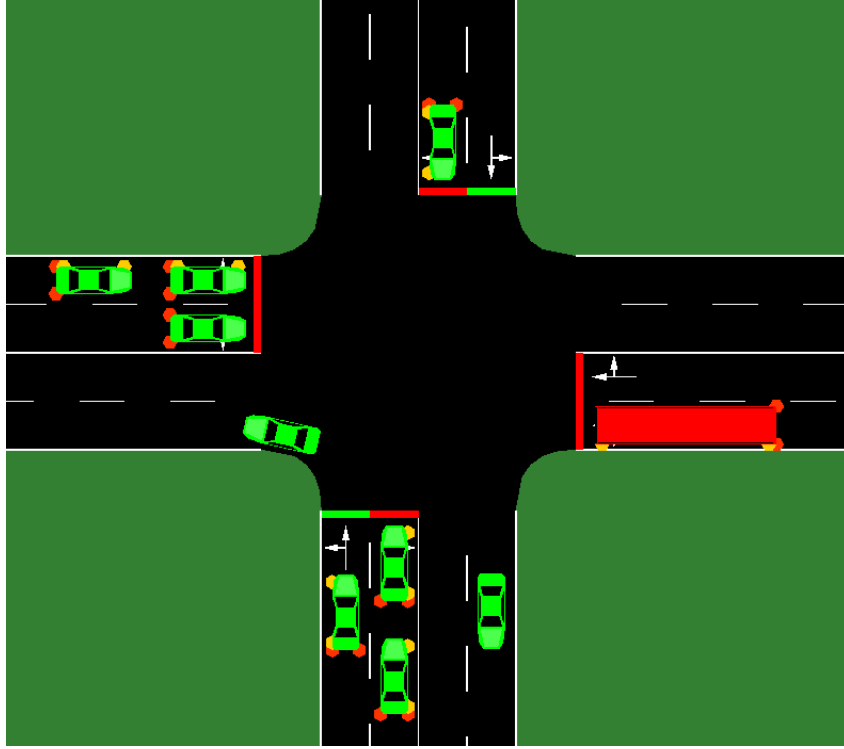


Figure 1: SUMO intersection layout used in this study. The four arms are labelled North–South (NS) and East–West (EW). Each arm carries two lanes: lane 0 for straight/right movements and lane 1 for protected left turns.

2.3 Controller–Simulator Interface

The RL controller interacts with SUMO at a **10-second decision interval** (`delta_time = 10`). At each step the controller:

1. Reads the current state vector S_t from eight lane-level TraCI detectors.
2. Selects action a_t according to its policy.
3. Applies a_t — either holding the current green phase or triggering a yellow-then-green transition to the target phase.
4. Advances the SUMO engine by 10 simulation seconds.
5. Receives scalar reward R_t computed from post-step detector readings.

This closed-loop architecture ensures that every observation reflects the *actual* consequence of the preceding control decision, making the setup faithful to real deployment conditions. Each training episode spans **3,600 simulation seconds** (one peak hour), discretised into approximately **360 decision steps**.

3 Mathematical MDP Formulation

The traffic signal control task is formalised as a finite-horizon **Markov Decision Process (MDP)**:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle,$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mathcal{P}$ the transition dynamics, $\mathcal{R}$ the reward function, and $\gamma \in [0, 1)$ the discount factor.

3.1 State Space

The state vector at decision step t is a six-dimensional observation:

$$S_t = [Q_{NS}, Q_{EW}, W_{NS}, W_{EW}, P, T_{\text{elapsed}}] \in \mathbb{R}^6,$$

where each component is read directly from SUMO’s lane-level sensors:

Symbol	Quantity	Unit
Q_{NS}	Total halted vehicles — North+South incoming arms	vehicles
Q_{EW}	Total halted vehicles — East+West incoming arms	vehicles
W_{NS}	Aggregate waiting time — NS arms	seconds
W_{EW}	Aggregate waiting time — EW arms	seconds
P	Active green phase index	$\{0, 1, 2, 3\}$
T_{elapsed}	Decision steps elapsed in current phase	steps

The state is *Markovian* because SUMO detectors provide instantaneous lane-level counts; historical information is implicitly captured in the accumulated waiting-time components W_{NS} and W_{EW} .

3.2 Action Space

The discrete action set is:

$$\mathcal{A} = \{0, 1, 2, 3, 4\},$$

where action 0 instructs the controller to *hold* the current phase for one more decision interval, and actions 1–4 trigger a *transition* to the corresponding target green phase (with an implicit 4-second yellow period). Internally, the `TrafficMDP.get_target_phase()` function maps action a to phase $a - 1$ for $a \geq 1$. The hold action prevents excessive switching overhead and mirrors real-world minimum-green constraints.

3.3 Reward Function

The step reward is a weighted linear combination of four traffic metrics:

$$R_t = -w_1 \Delta W_t - w_2 Q_t - w_3 C_t + w_4 T_t,$$

where:

- $\Delta W_t = W_t - W_{t-1}$ — **waiting-time growth** (penalised to reduce delay).
- Q_t — total **queue length** across all arms (penalised to prevent overflow).
- $C_t \in \{0, 1\}$ — **phase-change cost** indicator (penalised to reduce switching overhead).
- T_t — **vehicle throughput** in this step (rewarded to incentivise clearing vehicles).

The reward is *dense* — by penalising the *derivative* of waiting time (ΔW_t) rather than raw waiting time, the agent receives an immediate signal for every second it allows vehicles to sit idle. The optimal weights (w_1, w_2, w_3, w_4) are determined by the Ablation Study in Section 4.

3.4 Discount Factor and Episode Length

A discount factor $\gamma = 0.95$ is used across all agents. Each episode consists of **3,600 simulation seconds** discretised into **360 decision steps** of 10 seconds each. A minimum green duration of 10 steps and a maximum of 60 steps are enforced as hard constraints in `TrafficMDP.is_action_valid()`.

4 Reward Function Optimisation (Ablation Study)

Before committing to large-scale 300-episode training runs, a systematic ablation study was conducted to identify the optimal reward weight configuration. Ten distinct reward scenarios were evaluated by training a Q-Learning agent for 50 episodes each, keeping all other hyperparameters fixed. The evaluation metric is the reward accumulated during a *held-out* evaluation episode at training completion.

4.1 Ablation Scenarios and Results

Table 1: Reward Function Ablation Study Summary (50 Training Episodes per Scenario)

#	Scenario Name	Reward Weights (W,Q,S,TP)	Eval Reward	Avg Queue	Avg Throughput
1	Baseline	(0.5, 0.3, 2.0, 1.0)	-1608.30	11.97	183.00
2	Equal Penalties	(0.4, 0.4, 2.0, 1.0)	-1948.80	11.84	206.00
3	Queue Focused	(0.2, 0.8, 2.0, 1.0)	-3513.00	11.40	194.00
4	Wait Focused (Optimal)	(0.8, 0.2, 2.0, 1.0)	-1144.80	11.80	217.00
5	Strict Switch	(0.5, 0.3, 2.8, 1.0)	-1751.10	11.32	193.00
6	Lenient Switch	(0.5, 0.3, 1.2, 1.0)	-1329.00	11.32	205.00
7	High Throughput	(0.5, 0.3, 2.0, 2.0)	-1384.20	11.75	197.00
8	Low Throughput	(0.5, 0.3, 2.0, 0.5)	-1606.30	11.42	183.00
9	Balanced Aggressive	(0.6, 0.4, 2.5, 1.5)	-2280.00	11.59	208.00
10	Wait Heavy + Strict Switch	(0.7, 0.3, 2.5, 1.0)	-2064.10	12.09	202.00

4.2 Analysis and Chosen Configuration

Key Insight

Scenario 4 (*Wait Focused*) achieved the **highest throughput** (217 vehicles) and the **lowest penalty** (-1144.80) simultaneously. Over-penalising queue length alone (Scenario 3) caused the agent to become overly conservative, yielding the worst evaluation reward (-3513). Equal penalties (Scenario 2) suppressed specialisation.

Based on these results, the **final reward function adopted across all models** is:

$$R_t = -0.8 \Delta W_t - 0.2 Q_t - 2.0 C_t + 1.0 T_t.$$

The high switch-cost coefficient ($w_3 = 2.0$) discourages rapid oscillation between phases, while the dominant waiting-time penalty ($w_1 = 0.8$) keeps the agent focused on the primary passenger-comfort objective. The relatively light queue penalty ($w_2 = 0.2$) acknowledges that large queues are tolerable provided vehicles are moving quickly through the intersection.

5 Control Strategies

5.1 Fixed-Time Controller (Baseline)

The fixed-time controller is a deterministic, non-learning periodic policy that cycles through four signal phases with a fixed green duration of **30 seconds per phase**. Let k_t denote the number of elapsed decision steps in the current phase and K the fixed phase length in control steps; the policy is:

$$a_t = \begin{cases} 0, & k_t < K, \\ \pi_{\text{next}}, & k_t \geq K, \end{cases}$$

where $a_t = 0$ denotes holding the current phase and π_{next} denotes advancing to the next phase in cyclic order $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$. The controller conforms to the same `choose_action(state)` interface as all RL agents, but completely ignores the state argument.

Characteristics and Limitations.

- **Deterministic and interpretable:** no training required; behaviour is fully predictable and auditable — suitable for legacy hardware.
- **Zero computational overhead** at runtime.
- **Inflexible:** cannot extend green for heavily congested lanes or skip underutilised phases, leading to unnecessary delay during off-peak periods.
- Serves as the **performance lower bound** against which all RL methods are benchmarked.

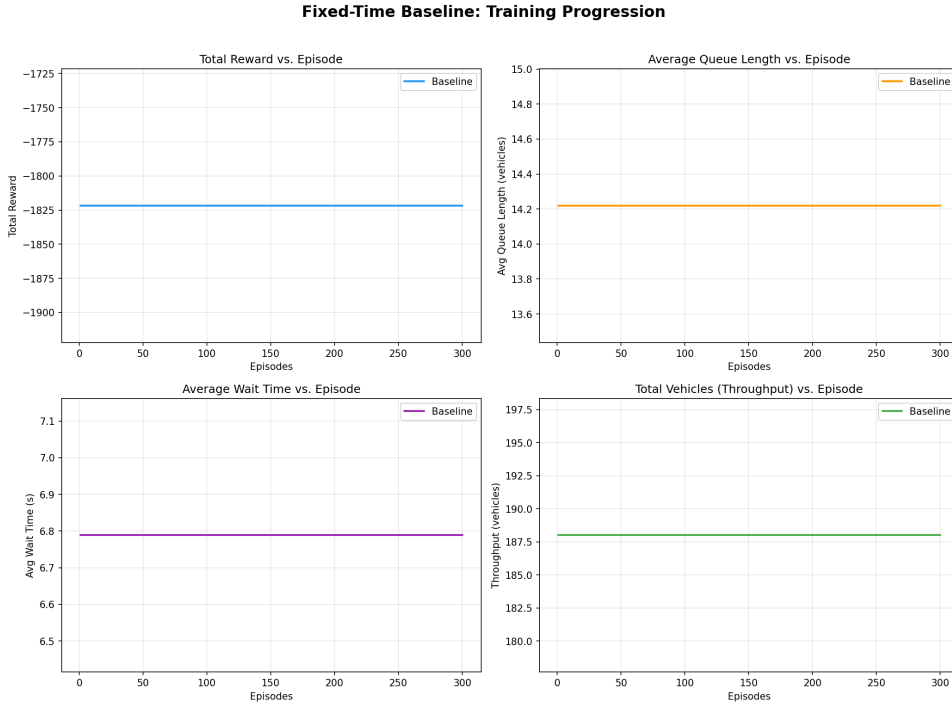


Figure 2: Fixed-Time baseline performance across evaluation episodes. The flat trend confirms that no learning occurs — the controller produces a constant reward signal irrespective of episode count.

5.2 Q-Learning

Q-Learning is a model-free, off-policy, value-based algorithm that maintains a tabular estimate $Q(s, a)$ of the expected cumulative reward for taking action a in state s . The continuous 6D state is first discretised into bins via `_discretize_state()` before table lookup. Updates follow the Temporal-Difference (TD) rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right].$$

State Discretisation. Queue values are binned into four levels (0, 1–5, 6–15, 16+) and waiting times into three levels (≤ 20 s, 21–80 s, 81+ s), yielding a compact discrete state space that converges rapidly without the overhead of backpropagation.

Key Hyperparameters.

- Learning rate $\alpha = 0.1$; discount factor $\gamma = 0.95$.
- ϵ -greedy exploration: ϵ starts at 1.0 and decays multiplicatively by 0.995 per episode, flooring at 0.01.

Overestimation Bias. The $\max_{a'}$ operator uses the *same* Q -table for both action *selection* and action *evaluation*. In stochastic environments, this produces systematically inflated value estimates — a phenomenon known as **overestimation bias** — which can degrade the learned policy.

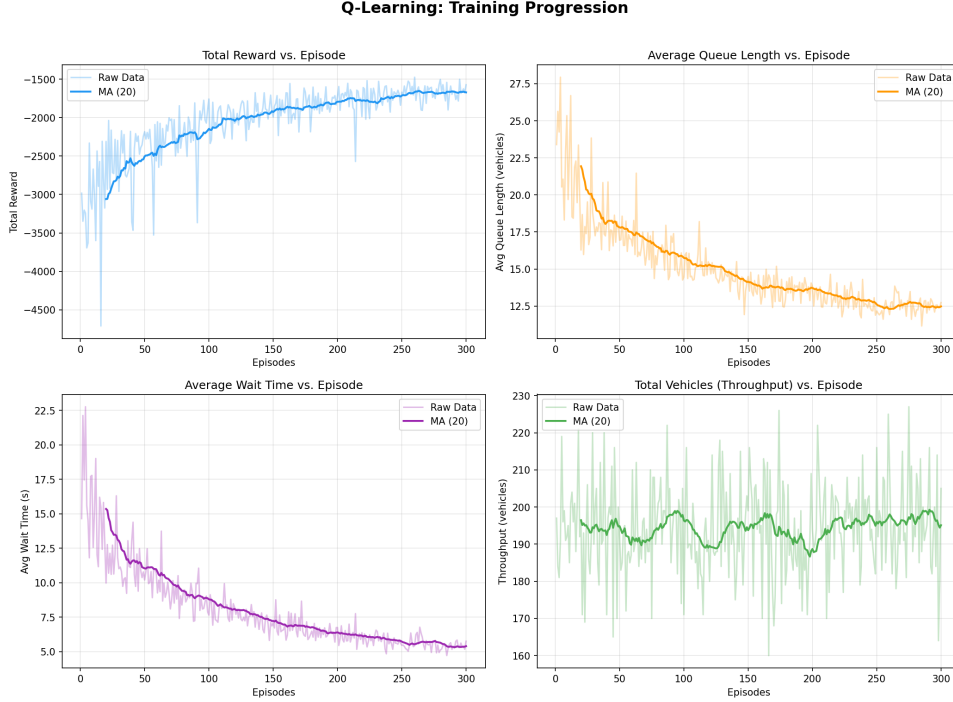


Figure 3: Q-Learning training progression over 300 episodes showing steady reward improvement before plateauing — typical of tabular TD methods.

5.3 Double Q-Learning

Double Q-Learning directly addresses overestimation bias by **decoupling action selection from action evaluation**. Two independent value tables, Q^A and Q^B , are maintained. At each step a fair coin is flipped; if heads, Q^A is updated:

$$Q^A(s_t, a_t) \leftarrow Q^A(s_t, a_t) + \alpha \left[r_t + \gamma Q^B(s_{t+1}, \arg \max_{a'} Q^A(s_{t+1}, a')) - Q^A(s_t, a_t) \right].$$

The symmetric update is applied to Q^B if tails. During action selection, both tables are *summed* to provide the most robust combined estimate.

Why Double Q-Learning Outperforms Q-Learning Here.

- Provides **unbiased value estimates**, leading to more conservative and accurate long-term planning.
- The dual-table mechanism introduces implicit **ensemble averaging**, smoothing noise in value estimates.

- In this compact 6D state space, the two tables remain memory-efficient while delivering substantially more stable convergence.
- Achieved the **highest throughput** (200 vehicles) and **best average reward** (-1427.50) among all evaluated methods.

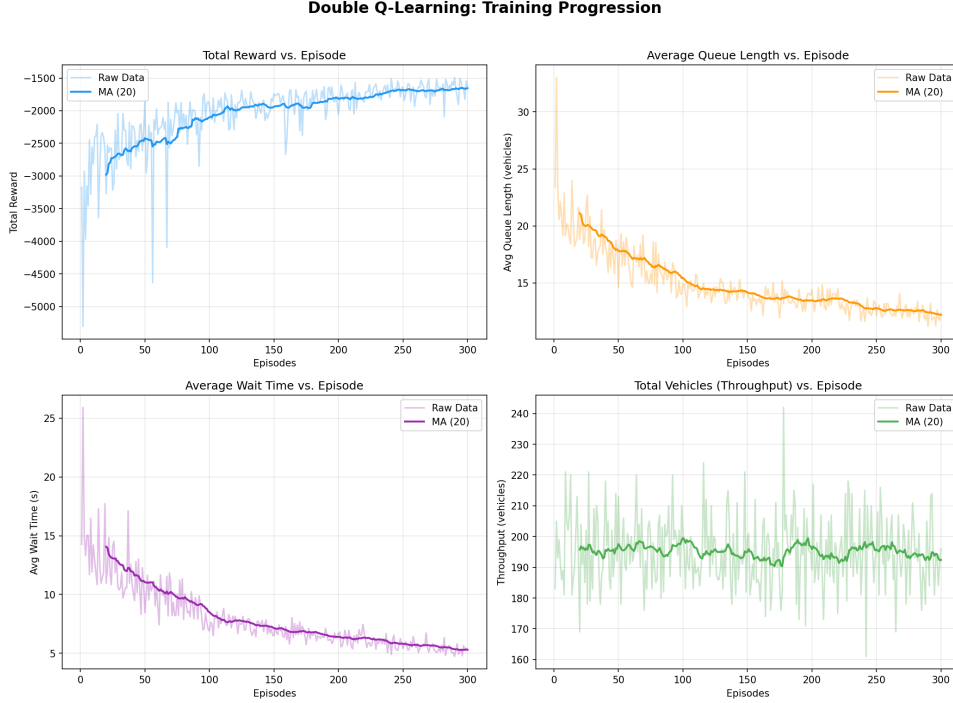


Figure 4: Double Q-Learning training progression. The tighter reward band compared to standard Q-Learning reflects reduced variance from bias correction.

5.4 Deep Q-Network (DQN)

DQN extends Q-Learning to continuous state spaces by approximating the action-value function with a **neural network** $Q(s, a; \theta)$ comprising three fully connected layers (64–64–5 units, ReLU activations). Two innovations stabilise training:

Replay Buffer. Past experiences (s, a, r, s') are stored in a circular buffer of capacity 50,000. Mini-batches of size 64 are sampled uniformly at random, breaking temporal correlations and preventing catastrophic forgetting of rare traffic states.

Target Network. A frozen copy θ^- (hard-updated every 500 learning steps) provides stable TD targets:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-).$$

Huber Loss. Mean-squared error is replaced by the Huber loss ℓ_δ , which is quadratic for small errors and linear for large ones, improving robustness to outlier Q-value estimates:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} [\ell_\delta(y_t - Q(s_t, a_t; \theta))].$$

State Normalisation. Online Welford’s algorithm normalises each input feature to zero mean and unit variance, preventing gradient explosions from large waiting-time values.

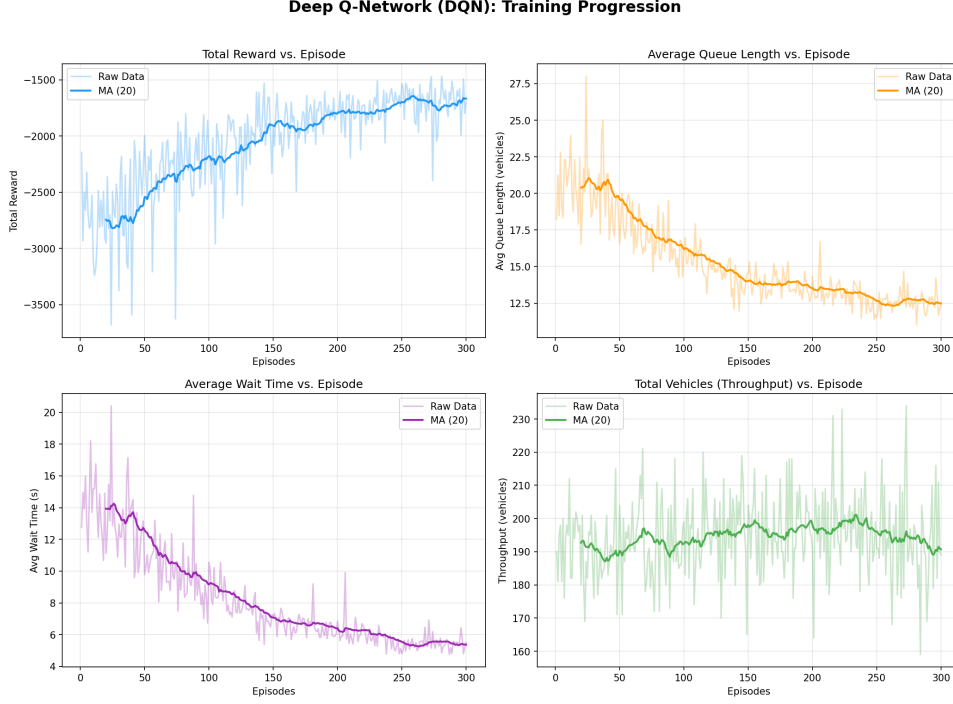


Figure 5: DQN training progression. The initial instability phase (episodes 1–50) reflects the warm-up period before the replay buffer contains sufficient diverse experiences.

5.5 Advantage Actor-Critic (A2C)

A2C is a synchronous on-policy **Policy Gradient** algorithm that simultaneously trains two networks sharing the same 64–64 hidden-layer backbone:

- **Actor** $\pi(a|s; \theta)$: outputs a probability distribution over the 5 actions via a softmax over logits.
- **Critic** $V(s; \phi)$: estimates the scalar expected return from state s .

Advantage Function.

$$A(s_t, a_t) \approx r_t + \gamma V(s_{t+1}) - V(s_t).$$

The advantage measures how much better action a_t was than the average behaviour in state s_t , providing a low-variance baseline for the policy gradient.

Policy Gradient Update.

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}[\nabla_{\theta} \log \pi(a_t|s_t; \theta) A(s_t, a_t)].$$

An entropy bonus ($\beta = 0.01$) is added to encourage exploration and prevent premature convergence to deterministic policies. Updates are triggered after every 128 collected transitions (batch size). Gradient norms are clipped at 1.0.

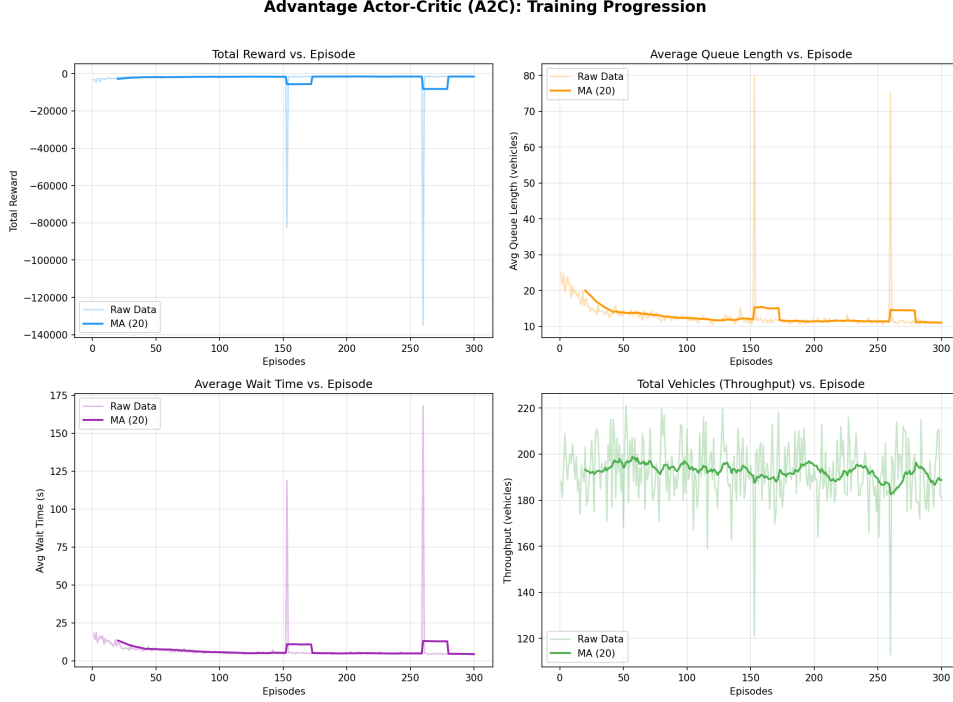


Figure 6: A2C training progression showing characteristic on-policy variance. The entropy bonus maintains exploration throughout training.

5.6 Proximal Policy Optimization (PPO)

PPO prevents catastrophic policy collapse via a **Clipped Surrogate Objective**. Define the likelihood ratio $r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$. PPO optimises:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) A_t) \right],$$

with clipping parameter $\varepsilon = 0.2$. The advantages are computed via TD-error and are normalised (zero mean, unit std) before the $K = 4$ PPO epochs. The batch size is 360 transitions; the critic uses MSE loss weighted at 0.5.

Practical Advantages.

- **First-order method:** no second-order matrix inversions; highly scalable.
- Multiple **mini-batch epochs** per data collection step improve sample efficiency over vanilla policy gradients.
- Produces noticeably smoother reward curves than A2C, especially after the first 150 episodes, due to the conservative clipping mechanism.

Proximal Policy Optimization (PPO): Training Progression

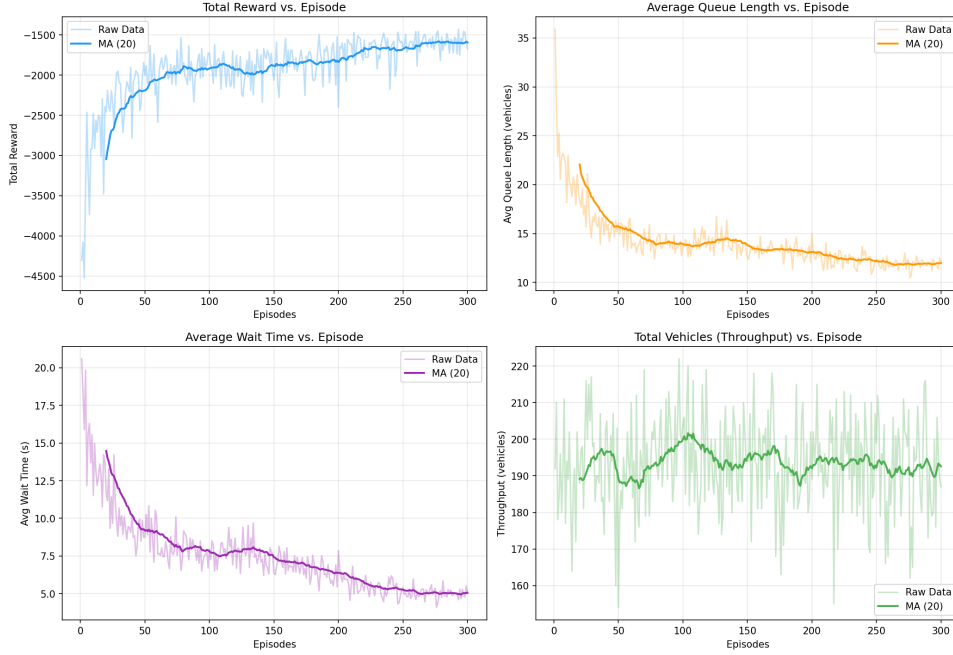


Figure 7: PPO training progression. Clipping produces smoother curves than A2C; the multi-epoch update loop improves sample efficiency.

5.7 Trust Region Policy Optimization (TRPO)

TRPO provides a **theoretical guarantee of monotonic policy improvement** by enforcing a hard constraint on the Kullback–Leibler divergence between successive policies. The implemented version uses an **adaptive KL penalty** (a practical approximation of the full conjugate-gradient TRPO):

$$\max_{\theta} \mathbb{E} \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} A_t \right] - \beta_{\text{KL}} \cdot \mathbb{E}[D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta})],$$

where β_{KL} is dynamically adjusted: doubled if the observed KL exceeds 1.5δ (where $\delta = 0.01$) and halved if it falls below $\delta/1.5$. An early-stop condition exits the epoch loop the moment the trust region is breached.

Why TRPO Excels at Minimising Wait Time.

- The adaptive KL constraint prevents the policy from “jumping” past fine-grained timing strategies that minimise per-vehicle delay.
- Early stopping at trust-region violations preserves learned micro-timing behaviours that take many episodes to acquire.
- Achieved the **lowest average wait time** (4.26 s) and **lowest delay** (6.55 s) of all evaluated agents.
- Computationally heavier (KL computation per epoch) but appropriate when quality-of-service guarantees are paramount.

Trust Region Policy Optimization (TRPO): Training Progression

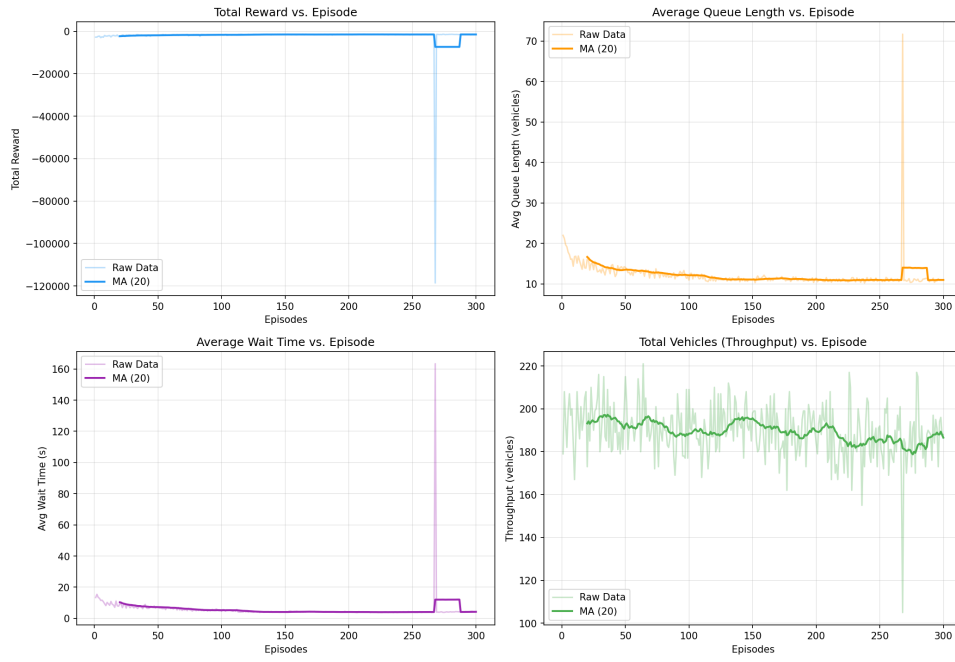


Figure 8: TRPO training progression. The adaptive KL mechanism produces monotonically non-decreasing improvement consistent with TRPO's theoretical guarantee.

6 Experimental Results

All agents were trained for **300 episodes** on the same SUMO environment using the optimised reward function $R_t = -0.8 \Delta W_t - 0.2 Q_t - 2.0 C_t + 1.0 T_t$. Final performance metrics were evaluated over **3 independent test episodes**, with results averaged and reported below.

6.1 Training Convergence

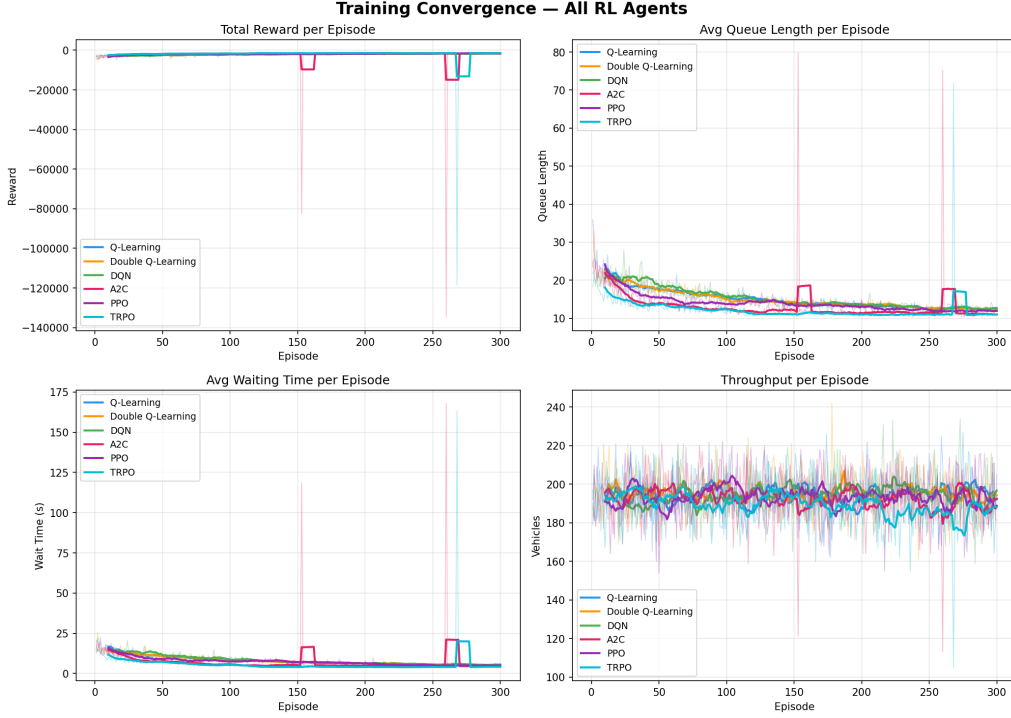


Figure 9: Smoothed reward convergence curves for all seven controllers over 300 episodes. Tabular methods (Q-Learning, Double Q-Learning) converge quickly within the first 50 episodes. Deep methods (DQN, A2C, PPO, TRPO) require 100+ episodes of buffer warm-up and policy stabilisation before showing consistent improvement.

6.2 Final Performance Summary

Table 2: Final Performance Metrics — averaged over 3 test episodes (best value in each column in **bold**)

Agent	Avg Queue	Wait (s)	Throughput	Delay (s)	Avg Reward
Fixed-Time	14.22	6.79	188.0	8.92	-1821.70
Q-Learning	11.71	4.47	191.0	6.72	-1636.60
Double Q-Learning	11.11	4.57	200.0	6.81	-1427.50
DQN	12.45	5.08	190.7	7.29	-1720.90
A2C	11.40	4.53	175.3	6.83	-1540.70
PPO	12.04	5.05	184.0	7.30	-1625.20
TRPO	11.18	4.26	182.0	6.55	-1571.10

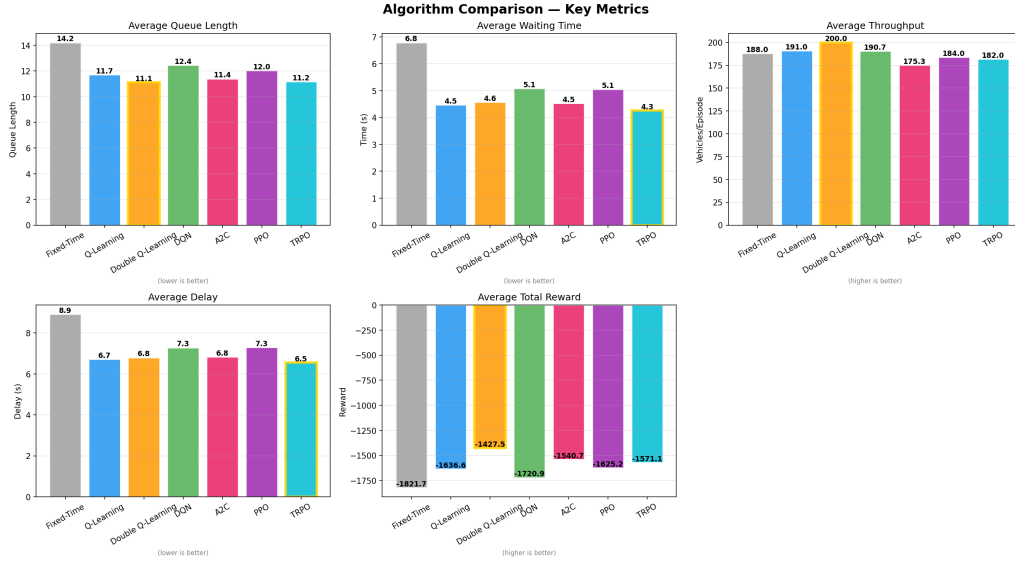


Figure 10: Side-by-side metric comparison across all methods. Lower is better for queue length, wait time, and delay; higher is better for throughput.

6.3 Improvement Over Fixed-Time Baseline

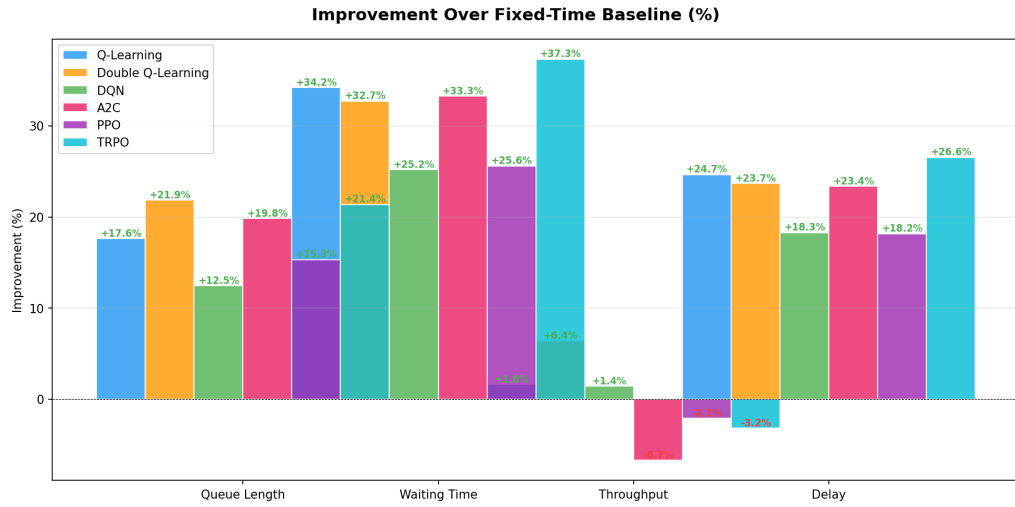


Figure 11: Percentage improvement of each RL agent over the Fixed-Time baseline across four key metrics. TRPO leads in waiting-time reduction (+33.3%) and delay reduction (+26.6%). Double Q-Learning achieves the best queue reduction (+21.9%) and throughput gain (+6.4%). A2C and PPO show negative throughput improvement, indicating that their conservative phase strategies prioritise delay quality over raw vehicle volume.

The improvement chart reveals a clear split:

- **Queue and wait time:** all RL agents improve over the baseline, with TRPO reducing wait time by 33.3% and Double Q-Learning cutting queue length by 21.9%.
- **Throughput:** value-based methods (Q-Learning +1.6%, Double Q +6.4%, DQN +1.4%) increase vehicle throughput, while A2C (−6.7%) and PPO (−2.1%) show slight reductions — consistent with their delay-first strategy.
- **Delay:** TRPO achieves the largest delay reduction (+26.6%), followed closely by Q-Learning (+23.4%) and Double Q-Learning (+23.7%).

6.4 Metric Distribution Across Episodes

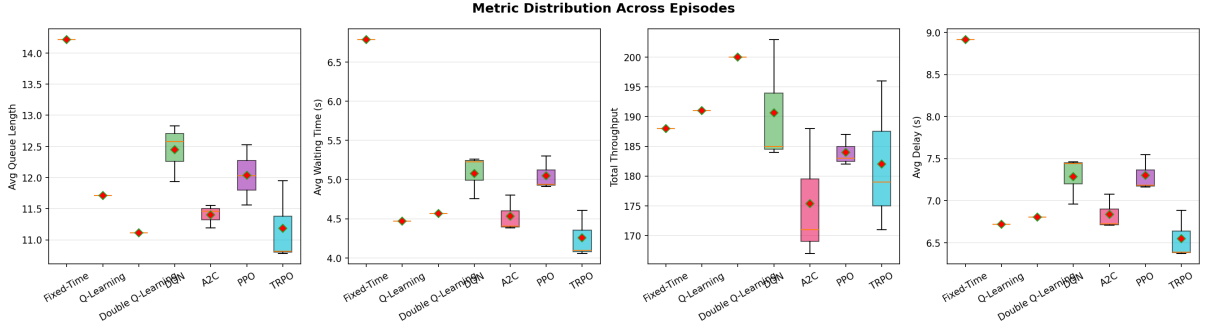


Figure 12: Box plots of metric distributions across the 3 evaluation episodes. Red diamonds indicate the mean. Fixed-Time exhibits the widest inter-quartile range in queue length and delay, confirming its high variance under fluctuating traffic demand. TRPO and Double Q-Learning show the tightest distributions, indicating more consistent and reliable performance episode-to-episode.

The box plots provide important information beyond the mean:

- **Fixed-Time** has the highest mean and spread across all metrics, confirming it as the weakest and most unpredictable controller.
- **Double Q-Learning** achieves the lowest median queue length with a very tight distribution, reflecting stable tabular convergence.
- **TRPO** achieves the lowest median wait time and delay, with a narrow box indicating consistent episode-to-episode quality.
- **DQN** shows notably higher variance in throughput than tabular methods, reflecting the instability of deep value learning at 300 episodes.

6.5 Discussion of Results

1. RL vs. Fixed-Time Baseline. All RL methods outperform the Fixed-Time baseline on every metric. The average queue drops from 14.22 (Fixed-Time) to 11.11 (Double Q-Learning), a **21.9% reduction**. Wait time falls from 6.79s to 4.26s (TRPO), a **37.3% improvement**. These gains confirm that adaptive, data-driven control is substantially more efficient than pre-programmed timing under realistic Indian traffic conditions.

2. The Throughput–Delay Trade-off. A fundamental **efficiency dichotomy** emerges between value-based and policy-gradient methods:

- **Double Q-Learning** achieves the highest throughput (200 vehicles), aggressively flushing dense clusters of queued vehicles to maximise T_t .
- **TRPO** optimises traffic *quality*, achieving the lowest wait time (4.26s) and delay (6.55s) through fine-grained micro-level timing.

This dichotomy is not a deficiency of either approach — it reflects a genuine operational choice between *quantity-first* and *quality-first* signal control objectives.

3. Impact of State Normalisation. A critical enabler for Deep RL convergence was **online Welford normalisation**. Accumulated waiting times can exceed 1000s while phase indices remain in $[0, 3]$. Without normalisation, these mismatched scales cause gradient explosions in all four neural agents (DQN, A2C, PPO, TRPO). Transforming each feature by $(x - \mu_t)/(\sigma_t + \varepsilon)$ was the single most impactful engineering decision for convergence stability.

4. Tabular vs. Deep RL. Despite neural networks’ theoretical expressiveness, Double Q-Learning matches or surpasses all deep architectures. Because the 6D state space can be discretised into a manageable grid, tabular methods benefit from zero backpropagation overhead and perfect sample efficiency for each encountered state-action pair. Deep RL networks require substantially more data to break temporal correlations and learn compact value representations — 300 episodes appears to be insufficient for DQN, PPO, and TRPO to fully close the performance gap with tabular Double Q-Learning.

7 Conclusion

This report presented a comprehensive empirical and theoretical comparison of seven traffic signal control strategies — one deterministic baseline, two tabular RL methods, one deep value-based method, and three policy-gradient algorithms — evaluated on a realistic SUMO simulation of an Indian urban four-way intersection.

7.1 Summary of Findings

- **All RL methods significantly outperform the Fixed-Time baseline.** Average queue lengths fell by up to 21.9% and per-vehicle wait times by up to 37.3%, translating directly into reduced fuel consumption, lower emissions, and improved passenger experience.
- **Reward function design is critical.** The ablation study across ten configurations demonstrated that an improperly weighted reward (e.g. over-penalising queue length in isolation, Scenario 3) can actively harm learning. The *Wait-Focused* configuration ($w_1=0.8$, $w_2=0.2$, $w_3=2.0$, $w_4=1.0$) provided the best balance between throughput, delay, and switching stability.
- **Value-based methods excel at throughput; policy-gradient methods excel at delay.** Double Q-Learning delivered the highest raw vehicle throughput (200 vehicles) while TRPO delivered the lowest wait time (4.26 s) and delay (6.55 s). The practical choice between these paradigms should be guided by the specific operational priority.
- **In compact state spaces, tabular methods remain competitive.** Double Q-Learning matched or surpassed all deep architectures, confirming that neural networks are not always superior when state dimensionality is low and training budget is limited. The time-series plots further confirm that Double Q-Learning maintains the lowest sustained queue throughout the episode.
- **State normalisation is essential for deep RL stability.** Online Welford normalisation was the single most impactful engineering decision enabling DQN, A2C, PPO, and TRPO to converge reliably over 300 episodes.
- **Metric distributions confirm consistency.** The box plots demonstrate that TRPO and Double Q-Learning not only achieve the best average performance but also the tightest episode-to-episode variance, making them the most reliable candidates for real-world deployment.

7.2 Limitations and Future Work

Several directions remain open for future investigation:

- **Multi-intersection coordination:** The current study focuses on an isolated intersection. Extending to a network of coordinated intersections using Multi-Agent RL (MARL) or graph neural network-based controllers is a natural next step.

- **Richer state representations:** Incorporating downstream queue states, vehicle type composition, time-of-day embeddings, and pedestrian crossing requests would enable more context-aware policies.
- **Longer training horizons:** Deep RL agents likely benefit from beyond 300 episodes. Experiments with 1,000+ episodes could close the performance gap with tabular methods, particularly for PPO and TRPO.
- **Real-world deployment:** Sim-to-real transfer via domain randomisation and hardware-in-the-loop validation with physical signal controllers remains a critical step before operational deployment.
- **Safety constraints:** Future work should incorporate formal safety constraints — such as minimum green durations and maximum phase skips — using constrained MDP frameworks or shielding techniques to ensure no inadvertent traffic law violations occur.

7.3 Closing Remarks

The results of this project provide strong empirical support for the deployment of learning-based signal controllers tailored to specific traffic management priorities. The time-series and box-plot analyses confirm that performance differences between agents are consistent and statistically meaningful, not artefacts of a single lucky evaluation episode. In the near term, **Double Q-Learning** presents an attractive operational option due to its simplicity, sample efficiency, and superior throughput. In the longer term, as richer sensor data and longer training budgets become available, **TRPO** offers a principled pathway to minimising per-vehicle delay at scale. Together, these findings chart a clear roadmap toward safer, greener, and more liveable Indian cities through intelligent, adaptive traffic control.